🏗️ System Architecture - Job Board Importer

Comprehensive system design and architecture documentation for the Job Board Importer application.

Table of Contents

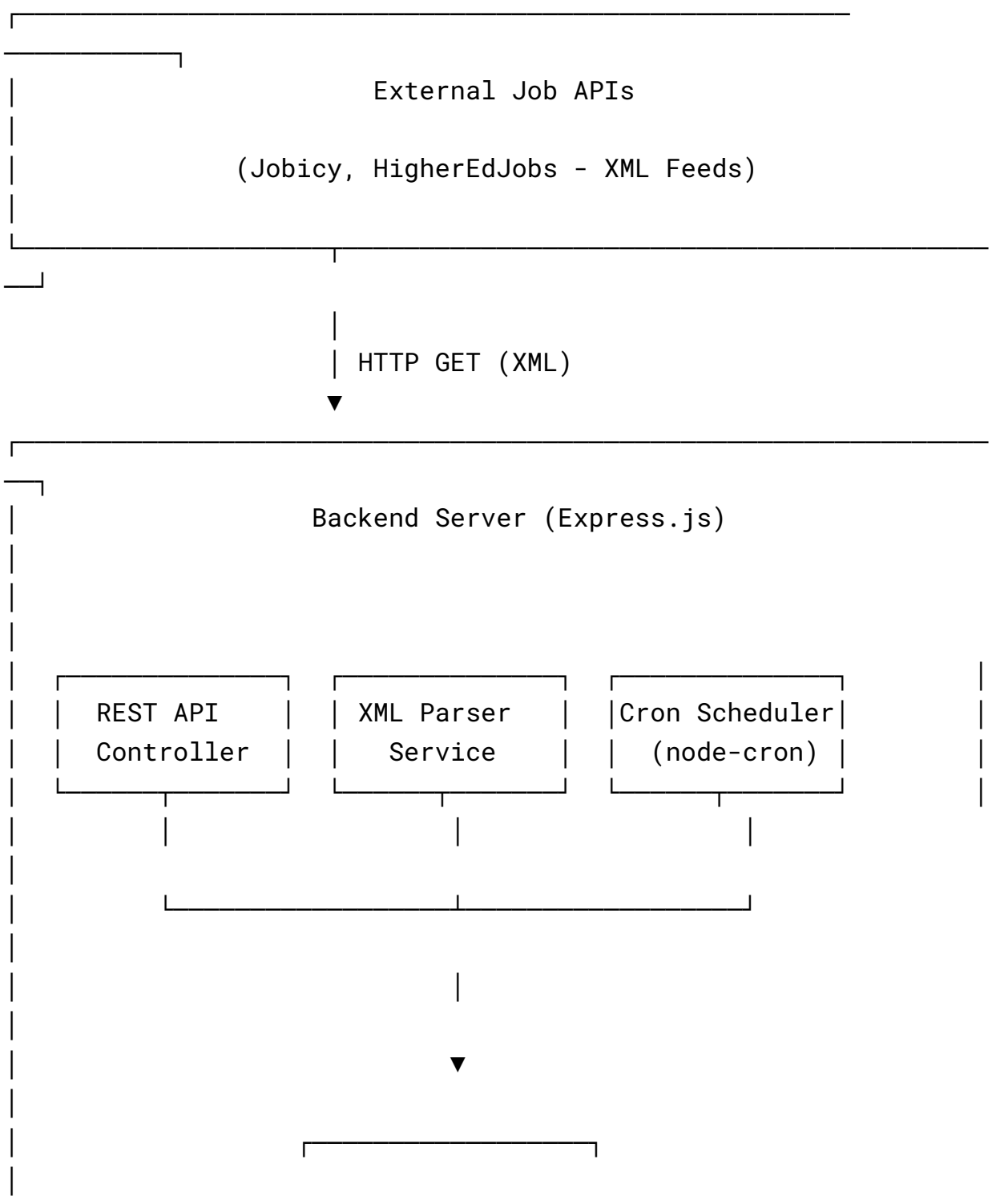
Overview

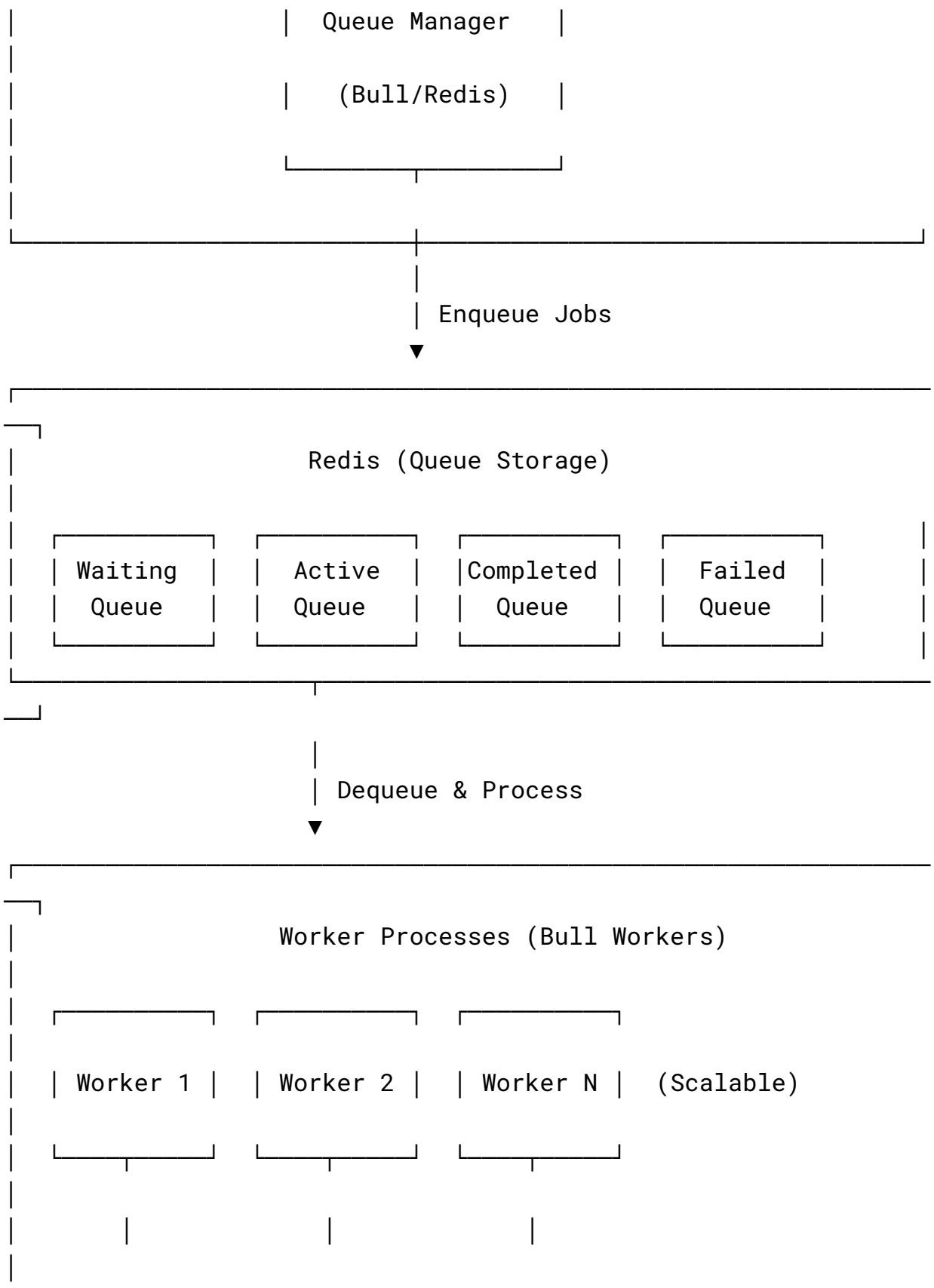
The Job Board Importer is a scalable, queue-based job import system built using the MERN stack (MongoDB, Express.js, React/Next.js, Node.js). The system fetches job listings from external XML APIs, processes them asynchronously using Redis-backed queues, and provides real-time progress tracking through Socket.IO.

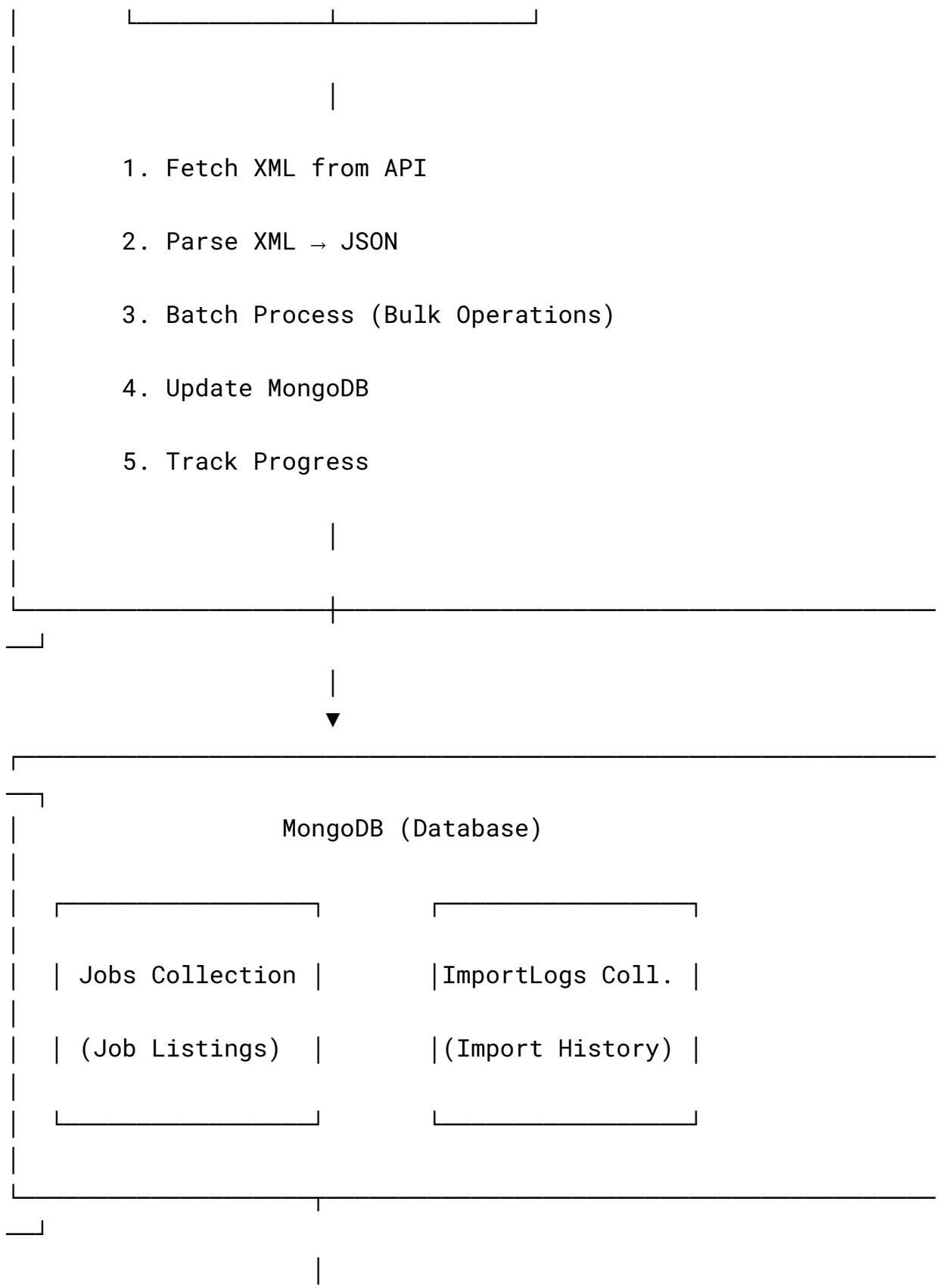
Key Characteristics

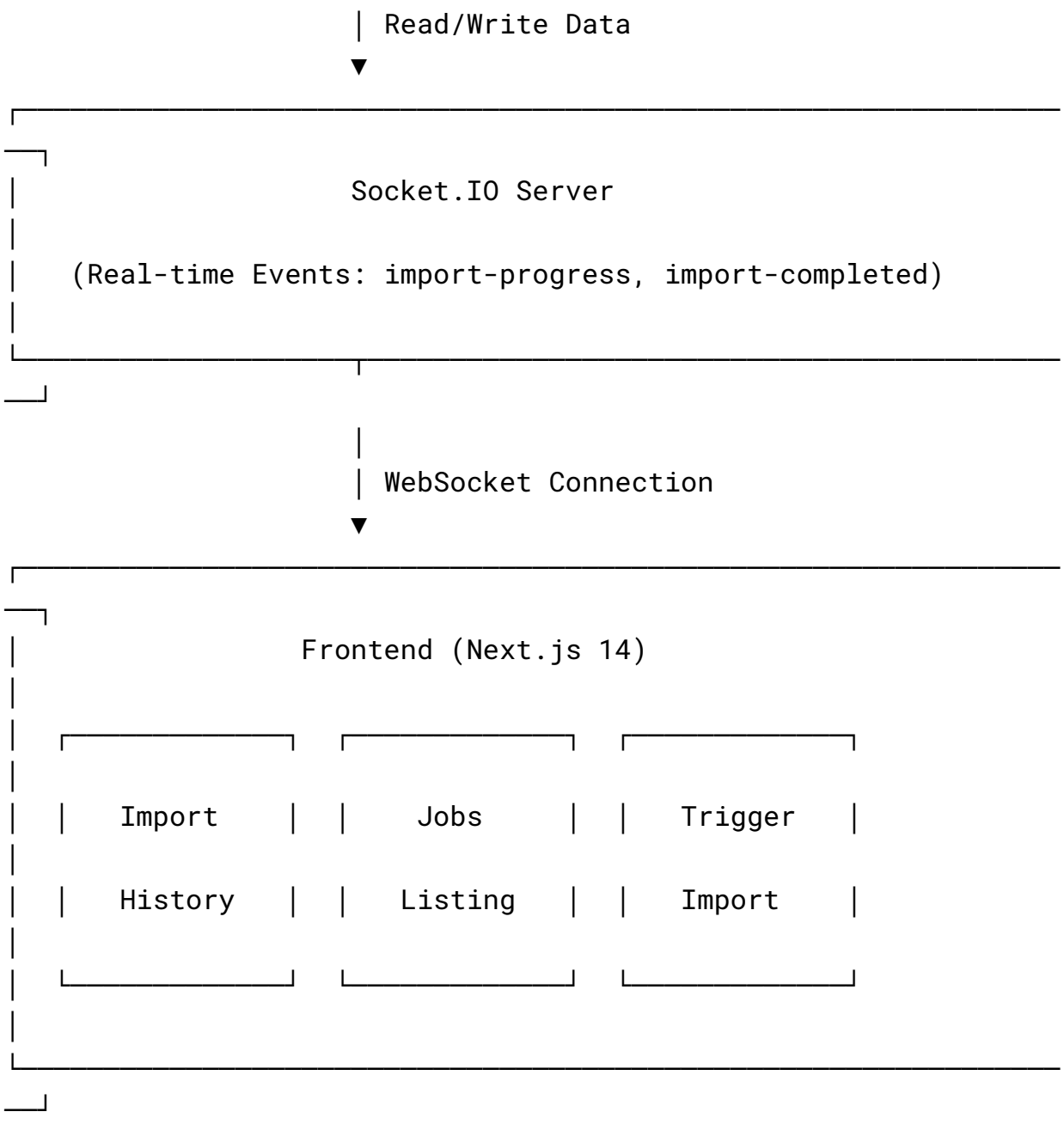
- Asynchronous Processing: Decoupled architecture using Bull queues
- Real-time Communication: Socket.IO for live updates
- Scalable Design: Horizontal scaling with multiple workers
- Fault Tolerant: Retry logic with exponential backoff
- Modular Structure: Clean separation of concerns

High-Level Architecture




Queue Manager
(Bull/Redis)
Enqueue Jobs
Redis (Queue Storage)
Waiting Queue
Active Queue
Completed Queue
Failed Queue
Dequeue & Process
Worker Processes (Bull Workers)
Worker 1
Worker 2
Worker N
(Scalable)


1. Fetch XML from API
2. Parse XML → JSON
3. Batch Process (Bulk Operations)
4. Update MongoDB
5. Track Progress
MongoDB (Database)
Jobs Collection
(Job Listings)
ImportLogs Coll.
(Import History)



Technology Stack

Backend Technologies

Technology	Version	Purpose
Node.js	18+	JavaScript runtime
Express.js	4.x	Web framework
MongoDB	6+	NoSQL database
Mongoose	8.x	MongoDB ODM
Bull	4.x	Queue management
Redis	3.0+	Queue storage
Socket.IO	4.x	Real-time communication
node-cron	3.x	Job scheduling
Axios	1.x	HTTP client

xml2js	0.6.x	XML parsing

Frontend Technologies

Technology	**Version**	**Purpose**
Next.js	14+	React framework
React	18+	UI library
Tailwind CSS	3.x	Styling
React Icons	5.x	Icon library
date-fns	4.x	Date formatting
Socket.IO Client	4.x	Real-time client

Component Design

1. Backend Architecture

A. API Layer (`src/routes/`, `src/controllers/`)

Responsibilities:

- Handle HTTP requests
- Route management
- Request validation
- Response formatting

Design Pattern: MVC (Model-View-Controller)

Key Routes:

javascript

```
POST   /api/imports/trigger     // Trigger manual import
GET    /api/imports/history     // Get import logs
GET    /api/imports/:id         // Get specific import
GET    /api/jobs                // Get jobs list
GET    /api/jobs/:id            // Get specific job
GET    /api/jobs/stats          // Get job statistics
```

B. Queue System (`src/config/queue.js`)

Responsibilities:

- Initialize Bull queue
- Configure job options
- Manage queue lifecycle

Configuration:

javascript

```
{
  attempts: 3,                    // Retry failed jobs 3 times
  backoff: {
    type: 'exponential',          // Exponential backoff
    delay: 2000                   // Start with 2s delay
  },
  removeOnComplete: 100,          // Keep last 100 completed
```

```
  removeOnFail: 500               // Keep last 500 failed
}
```

Why Bull (Not BullMQ)?

- ✅ Compatible with Redis 3.0+ (wider compatibility)
- ✅ Simpler API for basic queue operations
- ✅ Production-proven reliability
- ❌ BullMQ requires Redis 5.0+ (deployment constraint)

C. Worker Process (`src/workers/jobImportWorker.js`)

Responsibilities:

- Fetch jobs from queue
- Parse XML to JSON
- Batch processing
- Database operations (bulk upsert)
- Error handling and logging
- Progress tracking

Processing Flow:

```
text

1. Receive job from queue
2. Fetch XML from external API
3. Parse XML → JSON
4. Process in batches (configurable size)
5. For each job:
   - Check if exists (by externalId)
   - Insert new OR update existing
   - Track stats (new/updated/failed)
6. Update progress in ImportLog
7. Emit Socket.IO events
8. Mark job complete/failed
```

Optimization Techniques:

- Bulk Operations: `insertMany()` and `bulkWrite()` instead of individual inserts
- Batch Processing: Process 50 jobs at a time (configurable)
- Parallel Processing: Multiple concurrent workers
- Indexed Lookups: Fast existence checks using `externalId` index

D. XML Parser (`src/utils/xmlParser.js`)

Responsibilities:

- Fetch XML from external APIs
- Convert XML to JSON
- Transform data to internal schema
- Handle different XML structures
- Error detection

Supported XML Formats:

```
javascript

// RSS 2.0 (most common)
xmlData.rss.channel.item

// Atom feed
xmlData.feed.entry

// Alternative structures
xmlData.channel.item
xmlData.item
```

Data Transformation:

```
javascript

Raw XML → Parsed JSON → Internal Schema
{
  guid: "123",
  title: "Engineer",
  company: "Corp"
}
→
```

```
{
  externalId: "123",
  title: "Engineer",
  company: "Corp",
  category: "IT",
  jobType: "Full-time"
}
```

E. Cron Scheduler (`src/cron/scheduledImport.js`)

Responsibilities:

- Schedule automated imports
- Trigger hourly imports
- Manage multiple job sources

Schedule: `0 * * * *` (Every hour at minute 0)

Job Sources: Configured via `.env` (comma-separated URLs)

---

2. Frontend Architecture

A. Next.js App Router (`src/app/`)

File-based Routing:

```
text
```

```
/                    → Home page
/imports             → Import history
/imports/trigger     → Trigger import form
/jobs                → Jobs listing
```

App Router Benefits:

- Server-side rendering (SEO-friendly)
- Automatic code splitting

- Built-in optimization
- Nested layouts support

B. React Components (`src/components/`)

Component Hierarchy:

```
text
```

```
Navbar
  ├── Navigation links
  └── Branding

ImportTable
  ├── Table headers
  ├── Import rows
  │   ├── Status badge
  │   ├── Progress bar
  │   └── Stats columns
  └── Pagination

JobCard
  ├── Job title
  ├── Company info
  ├── Category badges
  └── External link
```

State Management:

- Local state with `useState`
- Real-time updates via Socket.IO hook
- API calls via Axios service

C. Socket.IO Integration (`src/hooks/useSocket.js`)

Custom Hook:

```
javascript
```

```
const socket = useSocket();

useEffect(() => {
  socket.on('import-progress', (data) => {
    updateProgress(data);
  });
}, [socket]);
```

Benefits:

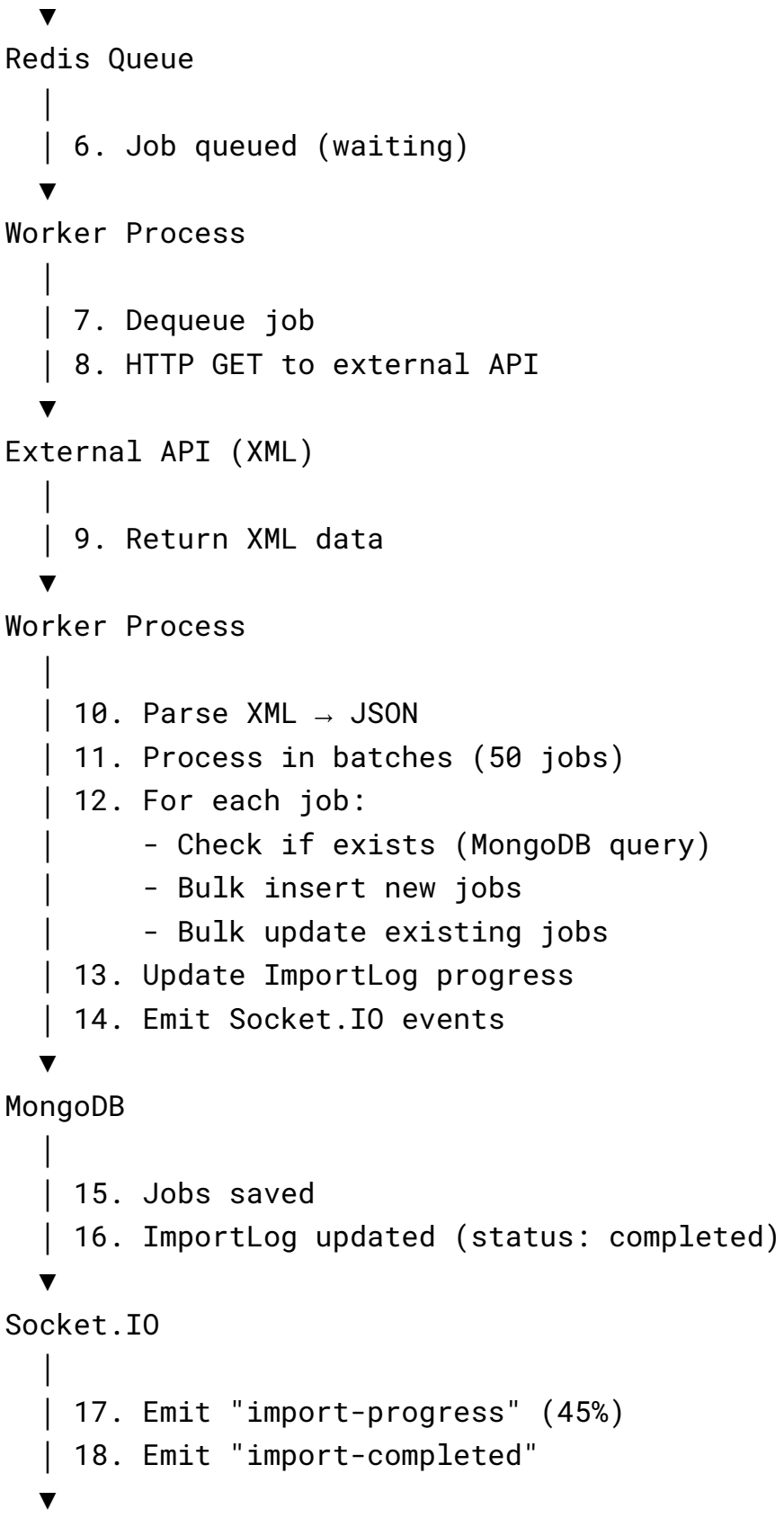
- Automatic reconnection
- Connection lifecycle management
- Event-driven updates
- Clean component integration

Data Flow

1. Manual Import Trigger Flow

```
text

User (Frontend)
  |
  | 1. Click "Trigger Import"
  ▼
Next.js (Client)
  |
  | 2. POST /api/imports/trigger
  |    Body: { url: "https://..." }
  ▼
Express API
  |
  | 3. Validate URL
  | 4. Create ImportLog (status: processing)
  | 5. Add job to Bull queue
```


Redis Queue
6. Job queued (waiting)
Worker Process
7. Dequeue job
8. HTTP GET to external API
External API (XML)
9. Return XML data
Worker Process
10. Parse XML → JSON
11. Process in batches (50 jobs)
12. For each job:
- Check if exists (MongoDB query)
- Bulk insert new jobs
- Bulk update existing jobs
13. Update ImportLog progress
14. Emit Socket.IO events
MongoDB
15. Jobs saved
16. ImportLog updated (status: completed)
Socket.IO
17. Emit "import-progress" (45%)
18. Emit "import-completed"

```
Frontend (Real-time)
  |
  | 19. Update progress bar
  | 20. Refresh import history
  | 21. Show notification
  |
  ✓ Complete
```

2. Scheduled Import Flow

```
text

Cron Job (Every Hour)
  |
  | 1. Trigger at 0 * * * *
  ▼
Scheduler
  |
  | 2. Read JOB_SOURCES from .env
  | 3. For each URL:
  |    - Create ImportLog
  |    - Queue import job
  ▼
[Same flow as Manual Import from step 6]
```

3. Job Search Flow

```
text

User
  |
  | 1. Enter search query
  | 2. Select filters
  ▼
Frontend
```

```
  |
  | 3. GET /api/jobs?search=...&category=...
  ▼
Express API
  |
  | 4. Build MongoDB query
  | 5. Apply filters
  | 6. Execute query (with indexes)
  | 7. Paginate results
  ▼
MongoDB
  |
  | 8. Return matching jobs
  ▼
Frontend
  |
  | 9. Display job cards
  | 10. Show pagination
  |
  ✓ Complete
```

Database Schema

Jobs Collection

```javascript
{
  _id: ObjectId,
  externalId: String,              // Unique (indexed)
  title: String,                   // Required
  company: String,
  location: String,
```

```
  description: String,
  url: String,
  category: String,                // Indexed
  jobType: String,                 // "Full-time" | "Part-time" |
etc.
  salary: String,
  postedDate: Date,
  sourceUrl: String,
  lastUpdated: Date,
  createdAt: Date,                 // Auto-generated
  updatedAt: Date                  // Auto-generated
}
```

Indexes:

javascript

```
{ externalId: 1 }                  // Unique index
{ category: 1, jobType: 1 }        // Compound index
{ postedDate: -1 }                 // Sorting index
```

ImportLogs Collection

javascript

```
{
  _id: ObjectId,
  fileName: String,                // URL identifier
  sourceUrl: String,
  status: String,                  // "processing" | "completed"
| "failed"
  totalFetched: Number,
  totalImported: Number,
  newJobs: Number,
  updatedJobs: Number,
  failedJobs: Number,
```

```
  failedReasons: [
    {
      jobId: String,
      reason: String,
      error: String,
      timestamp: Date
    }
  ],
  progress: Number,                  // 0-100
  duration: Number,                  // Milliseconds
  processedBy: String,               // Worker ID
  createdAt: Date,
  updatedAt: Date
}
```

Indexes:

```
javascript

{ status: 1, createdAt: -1 }       // Filter + sort
```

Technology Decisions

Why MERN Stack?

MongoDB:

- ✅ Flexible schema for varying job data structures
- ✅ Fast writes for batch imports
- ✅ Rich query capabilities
- ✅ Easy aggregation for statistics
- ✅ Horizontal scaling support

Express.js:

- ✅ Lightweight and fast
- ✅ Extensive middleware ecosystem
- ✅ Easy Socket.IO integration
- ✅ RESTful API support

React (Next.js):

- ✅ Component-based architecture
- ✅ Server-side rendering (SEO)
- ✅ Automatic code splitting
- ✅ Built-in optimization
- ✅ App Router for modern routing

Node.js:

- ✅ JavaScript everywhere (full-stack)
- ✅ Non-blocking I/O for async operations
- ✅ Large npm ecosystem
- ✅ Perfect for I/O-intensive tasks

Why Bull (Not BullMQ)?

Feature	**Bull**	**BullMQ**
Redis Version	3.0+ ✅	5.0+ ❌
API Complexity	Simple ✅	Advanced

Production Use	Proven ✅	Newer
Use Case	Basic queues ✅	Advanced features

Decision: Bull for wider Redis compatibility and simpler implementation.

Why Socket.IO?

- ✅ Real-time bidirectional communication
- ✅ Automatic reconnection
- ✅ Fallback to HTTP long-polling
- ✅ Event-driven architecture
- ✅ Easy to implement progress updates

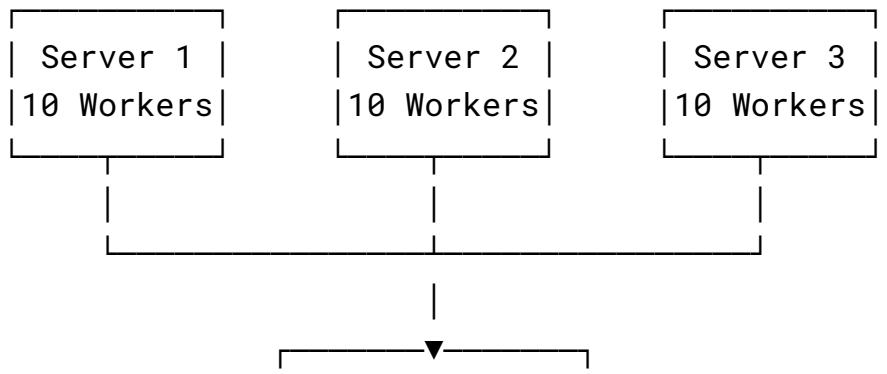
Scalability Considerations

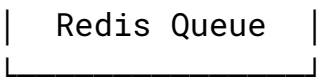
1. Horizontal Scaling

Worker Processes:

```
text
```

```
Current: Single server, 10 concurrent workers
Scaled:  Multiple servers, 10 workers each
```





Implementation:

- Deploy multiple worker instances
- Each connects to same Redis queue
- Bull handles automatic job distribution
- No code changes required

2. Database Scaling

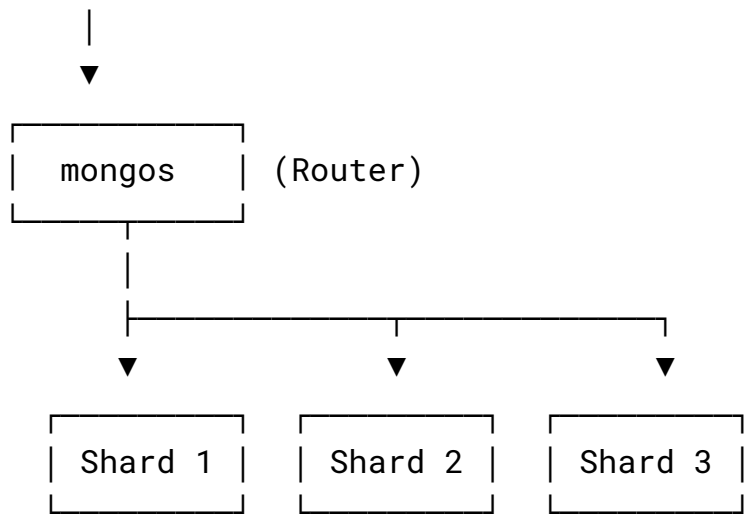
MongoDB Options:

Vertical Scaling:

- Increase RAM/CPU
- Use MongoDB Atlas M10+ tier

Horizontal Scaling (Sharding):

```
text
```



Shard Key: `category` (distribute by job category)

3. Redis Scaling

Options:

- Redis Cluster (multiple nodes)
- Redis Sentinel (high availability)
- Cloud Redis (Upstash, Redis Cloud)

Recommendation: Single Redis instance up to 10,000 jobs/hour

Performance Optimizations

1. Database Optimizations

Indexes:

```
javascript
```

```
// Critical for fast lookups
db.jobs.createIndex({ externalId: 1 }, { unique: true });
db.jobs.createIndex({ category: 1, jobType: 1 });
db.jobs.createIndex({ postedDate: -1 });
```

Query Optimization:

- Use `.lean()` for read-only queries (60% faster)
- Select only required fields
- Limit large queries
- Use aggregation for statistics

2. Batch Processing

Bulk Operations:

```
javascript
```

```
// ❌ Slow: Individual inserts
for (job of jobs) {
  await Job.create(job);
}
```

```
// ✅ Fast: Bulk insert
await Job.insertMany(jobs, { ordered: false });

// ✅ Fast: Bulk update
await Job.bulkWrite(updateOps, { ordered: false });
```

Performance Gain: 10-50x faster!

3. Concurrency Tuning

Configuration:

```
text

WORKER_CONCURRENCY=10    # Parallel jobs
BATCH_SIZE=50            # Jobs per batch
```

Tuning Guide:

- Low (1-2): Memory-constrained servers
- Medium (3-5): Balanced performance
- High (5-10): Powerful servers

4. Progress Update Throttling

```
javascript

// Update every 5 batches instead of every batch
if (i % (BATCH_SIZE * 5) === 0) {
  await ImportLog.findByIdAndUpdate(logId, { progress, stats });
}
```

Benefit: Reduces database writes by 80%

Security Considerations

Current Implementation

Implemented:

- ✅ CORS configuration (restrict origins)
- ✅ Input validation (URL validation)
- ✅ Error handling (no stack traces in production)
- ✅ Environment variables for secrets

To Implement (Production):

- ⚠️ Rate limiting (prevent API abuse)
- ⚠️ API authentication (JWT/API keys)
- ⚠️ Request size limits
- ⚠️ HTTPS only
- ⚠️ Helmet.js for security headers

Best Practices

1. Environment Variables:
    - Store all secrets in `.env`
    - Never commit `.env` to Git
    - Use different configs per environment
2. Data Validation:
    - Validate all user inputs
    - Sanitize data before database insertion
    - Use Mongoose schema validation
3. Error Handling:
    - Log errors server-side only
    - Return generic error messages to clients
    - Track failed operations in database

Deployment Architecture

Development Environment

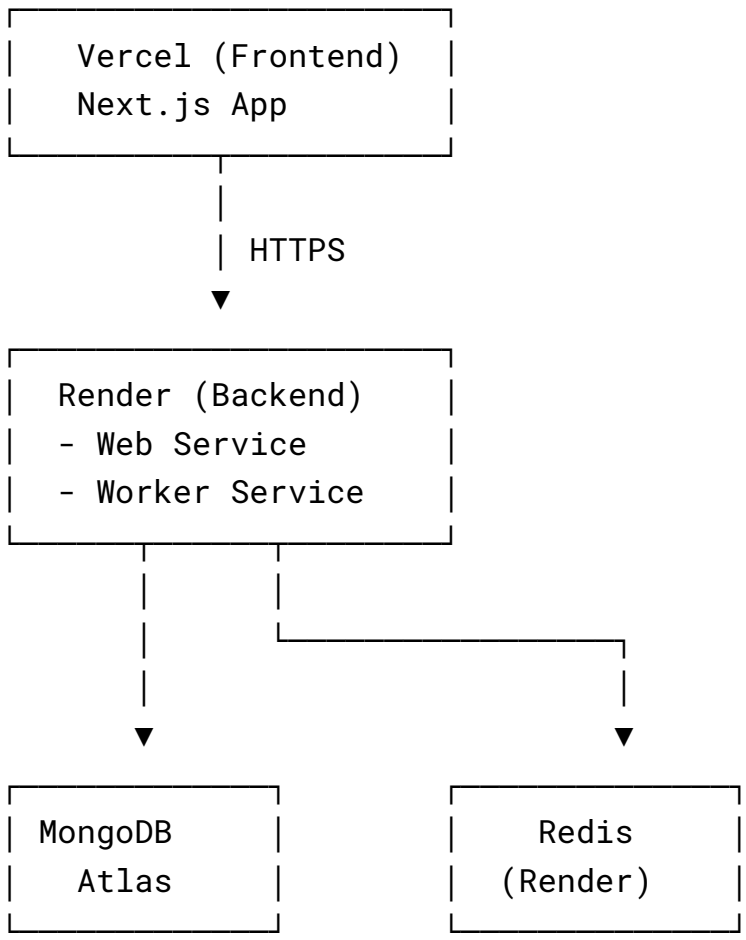
```
text
```

```
┌──────────────┐
│  Developer   │
```

```
|   Machine   |
|             |
| MongoDB     | (Local)
| Redis       | (Docker/Local)
| Node.js     |
| Next.js     |
└─────────────┘
```

Production Environment

text



Deployment Platforms:

- Frontend: Vercel (free tier, auto-deploy)
- Backend API: Render Web Service
- Worker: Render Background Worker
- Database: MongoDB Atlas (M0 free tier)
- Queue: Render Redis or Upstash

Performance Benchmarks

Import Speed

Optimization Level	Speed (1000 jobs)	Improvement
Original (sequential)	2-3 hours	Baseline
Batch processing	30-45 minutes	4-6x
Bulk operations	5-10 minutes	12-36x
Optimized (all)	2-5 minutes	24-90x

Database Operations

Operation	Without Index	With Index
Find by externalId	500ms	5ms

Filter by category	800ms	15ms
Sort by date	1200ms	20ms

Future Enhancements

Phase 1 (0-3 months)

- JWT authentication
- Rate limiting
- Advanced filtering
- Email notifications

Phase 2 (3-6 months)

- Job application tracking
- Analytics dashboard
- User profiles
- Saved searches

Phase 3 (6-12 months)

- Machine learning recommendations
- Duplicate detection
- Multi-tenancy support
- Microservices migration

Conclusion

This architecture provides:

✅ Scalability - Horizontal scaling via workers
✅ Reliability - Retry logic, error tracking
✅ Performance - Bulk operations, batch processing, indexes

✅ Maintainability - Modular design, clean code
✅ Real-time - Socket.IO for live updates
✅ Extensibility - Easy to add new features

Trade-offs Made:

- Bull over BullMQ (Redis 3.0 compatibility)
- MongoDB over SQL (flexible schema)
- Monolith over microservices (simpler deployment)

Success Metrics:

- Import 1000 jobs in < 5 minutes
- Support 10+ concurrent imports
- 99.9% uptime
- < 500ms API response time

Document Version: 1.0
Last Updated: October 18, 2025
Author: Afzal Vepari
GitHub:https://github.com/afzalveparii/Job-Board-Importer---Scalable-MERN-Stack-Application